Curriculum Comparison: Paid, Free, and What We Built

A look at the best-regarded paid high school CS programs, the strong free ones, and how our assembled course compares. Includes syllabus links and an honest account of what those programs contain that ours does not, and why.

Verify current prices and syllabus links before relying on them; costs and course editions change.

The programs, at a glance

Program	Cost	Language	Structure	AP-endorsed	Best at
CompuScholar	About $195 per course per year (or $25 per month) for most courses; the Computer Science Foundations (AP CSP) course is $225 per year. Verified August 2026	Python, Java, C#, web	Video lessons, auto-graded, transcript-ready, self-paced	Yes (CSP)	Turn-key homeschool credit with grading done for you
CodeHS	Free tier; paid Pro for grading and tools	Python, JavaScript, Java	Browser IDE, auto-grading, 60+ courses	Yes (CSP and CSA)	One integrated platform: lessons, IDE, grading
Project STEM	Free (sponsored)	Python (CSP), Java (CSA)	Video, auto-graded, teacher resources	Yes (CSP and CSA)	Free, rigorous, real AP prep with a proven track record
UTeach CS Principles	Paid per student	Python and blocks	Project-based, College Board syllabus	Yes (CSP)	Engaging, project-driven AP CSP
Harvard CS50 / CS50x	Free	C, Python, SQL, JS	Lectures plus problem sets	No (not AP)	Depth and intellectual heft; college-level
Code.org CS Principles	Free	App Lab (JavaScript)	Lessons, assessments, AP prep	Yes (CSP)	Free AP-aligned concepts and assessments
Our course	Free tools plus small costs	Python, real hardware	Instructor-led, systems narrative, hands-on labs	Aligned to CSP framework	Systems-level "how computers actually work," hands-on

Syllabus and course links

- CompuScholar course pages, each with a Course Syllabus tab: Python `compuscholar.com/homeschool/courses/python`, CS Foundations (AP CSP) `compuscholar.com/homeschool/courses/csfoundations`, Tech Essentials `compuscholar.com/homeschool/courses/tech-essentials`. The Java and AP scope-and-sequence documents are linked from their course pages.

- CodeHS: `codehs.com` (browse the course catalog; each course lists its syllabus).
- Project STEM AP CSP: `projectstem.org/high-school/ap-csp` and AP CSA at `projectstem.org`.
- UTeach CS Principles, with the College Board-approved syllabus: `cs.uteach.utexas.edu/csp`.
- Harvard CS50: `cs50.harvard.edu` (CS50x on edX is the free self-paced version).
- Code.org CS Principles: `https://code.org/en-US/curriculum/computer-science-principles` (Code.org now operates as CodeAI; older `code.org/educate/csp` links redirect here).
- The authoritative reference for any AP CSP alignment is the College Board Course and Exam Description: `apcentral.collegeboard.org/courses/ap-computer-science-principles`.
- Endorsed AP CSP providers list: `apcentral.collegeboard.org/courses/ap-computer-science-principles/classro`

How our course compares

Our course was built for a specific situation that none of the paid programs target: a small, in-person, mixed-age co-op taught by a technical instructor, organized around the narrative "how does a button press become something useful," with real hardware and unplugged activities, and programming depth in Python. The paid and free programs are, by contrast, mostly self-paced online platforms designed to run with little instructor expertise.

The trade-off is clear. Those platforms hand you auto-grading, a browser IDE, a ready transcript, and a College Board endorsement. Ours gives you a hands-on, systems-level experience and a coherent story, but you do the grading and you own the AP alignment yourself.

What those programs include that ours does not, and why

This is the useful part: the specific things a well-loved paid or endorsed program has that we deliberately left out or handled differently.

Auto-grading and a built-in IDE

CodeHS, CompuScholar, and Project STEM run code in the browser and grade it automatically, and CompuScholar explicitly requires no local install or file management. We deliberately went the other way: students install Python locally, use Thonny and then VS Code, and manage real files. For your goals (they understand what they built, and they learn the file system and terminal) local tooling is the right call, but it means you grade by hand and troubleshoot setups. This is a genuine convenience we gave up on purpose.

A formal AP Course Audit endorsement

CompuScholar, Project STEM, UTeach, and Code.org are College Board-endorsed, so a family can list an approved provider on the AP Course Audit and the syllabus is pre-approved. Ours is aligned to the framework but not audited. We chose framework alignment because you are not required to run an official AP course for a homeschooler to sit the exam, and because you wanted programming depth over teaching-to-the-exam. If audit-backed "AP" on the transcript matters, adopting one of these as the AP spine is the fix.

The Create Performance Task, fully scaffolded

The endorsed programs walk students through a mock Create Task, then a checklist and template for their own, with the submission timeline built in. We reference the Create Task and its 9 hours and the late-April deadline, but we did not build the scaffolding. This was scope: our depth-first design assumes you either use Code.org's Create Task materials or lean on an endorsed provider for the AP-track students. If you commit students to the exam, borrow that scaffolding rather than writing it.

Java and AP Computer Science A

CompuScholar, CodeHS, and Project STEM all offer a full Java-based AP CSA course. We intentionally omitted Java and CSA. CSA is a second, heavier year of pure programming, and it did not fit a broad, first-year, systems-oriented course for mixed ages. We noted CSA as a natural year-two target for a student who catches fire, which is the right place for it.

A pre-built bank of auto-graded exercises and tests

These platforms ship hundreds of graded problems and unit tests. We provide labs, checkpoints, projects, and a grading rubric, but not a large auto-graded item bank. For a small in-person class this is less necessary (you see the work directly), and Code.org's free assessment bank can fill the gap for AP-style questions. Still, it is real content we do not hand you ready-made.

Turn-key structure for a non-technical parent

The paid programs are explicitly designed so a parent without a CS background can run them, with student support from the vendor. Ours assumes a technical instructor, which you are. That assumption is what lets ours go deeper on systems and hardware than any of them, but it would not transfer to a non-technical teacher.

What our course has that the others do not

For balance, the things our course does that the paid and free platforms generally do not:

- A single systems narrative from transistors to cloud AI, rather than a topic checklist.
- Real hardware: the Apple IIe, a disassembled PC, a physical network, and the Raspberry Pi, which doubles as the class server later in the year.
- Unplugged, offline logic activities woven through the year.
- The build-the-stack arc (logic gates to a CPU to a tiny language) via Turing Complete and nand2tetris.
- A deliberate no-AI-until-taught policy so student work is genuinely theirs.
- Programming depth in local Python with real file and terminal skills, not a sandboxed browser IDE.

Recommendation

Keep our course as the spine, since it fits your setting and goals in a way no off-the-shelf program does. Then bolt on the two things the endorsed programs do better, using free options:

1. Use Code.org CS Principles (free) for AP-aligned concept lessons, the assessment bank, and Create Task scaffolding for any AP-track students.
2. If a family wants an audit-backed transcript or a fully turn-key fallback, Project STEM (free) or CompuScholar (paid; the AP CSP course is about $225 per year, most other courses about $195) is the one to adopt for that student, run alongside our course.

That gives you the depth and hands-on experience of what we built, with the AP safety net and grading convenience of the best platforms, at little or no added cost.